P2248R8
Enabling list-initialization for algorithms

Published Proposal, 2024-03-20

Author:
Giuseppe D'Angelo

Audience:
LWG, LEWG, SG6, SG9

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

We propose to default some template type parameters in certain Standard Library function templates, so that these functions become callable using list-initialization. These functions are algorithmic related functions, available in `<algorithm>` or in standard containers' headers.

Table of Contents

§ 1. Changelog

- R8
  - Wording-only changes after LWG review:
    - Rebase on top of the latest Standard draft.
    - Incorporate the changes to `std::ranges::fold` and `std::ranges::contains` (the respective papers have been merged after R7).
    - Fix a typo in the specification of `std::ranges::count` (missing comma).
    - Fix the reference to the exposition-only alias template `indirect-value-t`.
- R7
  - New revision incorporating the feedback from LEWG in Kona; see § 5.7 Is the Standard Library allowed to re-order deducible template parameters of algorithms?.
  - Rebased the wording on top of the latest C++ Standard draft.
  - Added freestanding comment to `projected_value_t`.
  - Fixed typos in the proposed wording.
- R6
  - Removed the numeric algorithms from the present proposal (see § 5.2 Numeric algorithms).
- R5
  - New revision incorporating the feedback after a LEWG telecon.
  - `projected_value` has been given a new, simpler formulation; and renamed to `projected_value_t`.
  - Discussed the interactions of `projected_value_t` with [P2609R0] (see § 5.9.1 Providing the projected value as an API).
  - Extended the proposal to also cover `ranges::fold` ([P2322R6]) and `ranges::contains` ([P2302R4]), should they get approved and merged.
  - Added a reference to the implementation experience in HPX.
- R4
  - Dropped the proposed default from the non-range version of `replace_copy`, elaborating on the rationale (see § 5.1.4 The problem of establishing a default for replace_copy).
  - Fixed minor misspellings.

- Clarified the wording for some design decisions regarding the numeric algorithms (no design has actually changed).

- R3
  - Fixed the default for the range version of `replace_copy_if`. Thanks to Casey Carter for spotting it.
  - Discussed the problem with projections in range-based algorithms.
- R2
  - Elaborated on the rationale.
  - Elaborated on [algorithms.requirements] and on [SD-8].
  - Elaborated on defaulting value types for algorithms that use output iterators.
  - Rebased on the top of the latest Standard.
  - Added a link to a prototype implementation.
  - Fixed the defaults for the `*_scan` numeric algorithms.
- R1
  - Clarified some possible sources of ABI incompatibility, which were (guiltly) not discussed in R0.
  - Added a reference and some discussion related to [P1997R1].
  - Added a reference to [algorithms.requirements].
  - Fixed misspellings / formatting.
- R0
  - First submission

§ 2. Tony Tables

Before	After

`struct point { int x; int y; };`
 	
`std::vector<point> v;`
 	
`v.push_back({3, 4}); // No need to specify the "point" type here...`
 	
`// ... but must spell out the type here:`
`// an initializer list in algorithms is not able to deduce the type`
`std::find(v.begin(), v.end(), point{3, 4});`
`std::lower_bound(v.begin(), v.end(), point{3, 4});`
`std::search_n(v.begin(), v.end(), 10, point{3, 4});`
`std::ranges::fill(v, point{3, 4});`
`erase(v, point{3, 4});`
	`struct point { int x; int y; };`
	
	`std::vector<point> v;`
	
	`v.push_back({3, 4});`
	
	`// With the proposed changes: car`
	`// the type gets deduced from the`
	`std::find(v.begin(), v.end(), {3,`
	`std::lower_bound(v.begin(), v.enc`
	`std::search_n(v.begin(), v.end(),`
	`std::ranges::fill(v, {3, 4});`
	`erase(v, {3, 4});`

§ 3. Motivation and Scope

List-initialization ([dcl.init.list]) has been introduced in C++11. When an initializer list is used as a function argument, overload resolution works in a specific way to find a suitable overload, possibly constructing a temporary object of the argument's type (see [over.ics.list]).

This language feature allows to write code like this:

```
void f(std::complex<double>);
f({1.23, 4.56}); // OK
```

In case of function templates, the deduction rules do not allow deduction in the general case, because a type cannot be determined ([temp.deduct.call]). It is however allowed to use an initalizer list in case the corresponding template type parameter has a default:

```
template <typename T> void g(T);
g({1, 2});     // ERROR: cannot deduce T


template <typename T = std::complex<double>> void h(T);
h({1.0, 2.0}); // OK, the default is used for deducing T
```

Default template type parameters have a somehow poor adoption in the Standard Library. They are used for instance in `std::exchange`:

```
namespace std {
  // [utility.exchange]
  template<class T, class U = T>
  constexpr T exchange(T& obj, U&& new_val);
}

// thanks to U defaulted to T, we can write this:
std::complex<double> old_val;
auto new_val = std::exchange(old_val, {3.0, 4.0}); // OK

// old_vec is moved onto new_vec, and reset to a default-constructed state
std::vector<int> old_vec;
auto new_vec = std::exchange(old_vec, {}); // OK
```

Unfortunately, they are not used in a number of other places, such as some standard algorithms, although reasonable defaults could still be determined based on other available template parameters.

Let's consider C++20's `std::erase` for `std::vector`:

```
namespace std {
  // [vector.erasure]
  template<class T, class Allocator, class U>
  constexpr typename vector<T, Allocator>::size_type
  erase(vector<T, Allocator>& c, const U& value);
}

std::vector<std::complex<double>> numbers;
numbers.push_back({1.0, 2.0}); // OK
erase(numbers, {1.0, 2.0});    // ERROR: cannot deduce U
```

Since `U` does not have a default, one cannot use list initialization for the `value` parameter (see the examples in the § 2 Tony Tables above). Instead, `U` could be reasonably defaulted to `T` in `erase`'s template argument list (just like `std::exchange`) and therefore enabling the above syntax. This is one of the changes we are proposing.

Similarly, an algorithm like `std::find` is also lacking a default:

```
namespace std {
  // [alg.find]
  template<class InputIterator, class T>
  constexpr InputIterator
  find(InputIterator first, InputIterator last, const T& value);
}

std::vector<std::complex<double>> numbers;
numbers.push_back({1.0, 2.0});                        // OK
std::find(vector.begin(), vector.end(), {1.0, 2.0}); // ERROR
```

In this case `T` could be reasonably defaulted to be the value type of the input iterator, that is, `typename std::iterator_traits<InputIterator>::value_type`. We are also proposing this change.

> In a nutshell: we are proposing to add a default template type parameter to the "algorithmic" function templates in the Standard Library:
>
> 1. which have a "value"-like function argument; and
> 2. for which a reasonable default type can be inferred from the other template type parameters.
>
> This enables users to call these functions by using list-initialization for the "value"-like arguments.

§ 4. Impact On The Standard

The proposed changes only affect existing Standard Library function templates. The impact is mostly positive: syntax that was ill-formed before becomes well-formed. There is a chance of incompatibility w.r.t. user code, but [SD-8] allows the kind of changes we are proposing.

This proposal does not depend on any other library extensions.

This proposal does not require any changes in the core language.

The first part of [P2218R0] is related to this proposal. [P2218R0] proposes to default the template type parameter of `optional::value_or`. We are not proposing the same, as we are limiting the scope of this paper to algorithms; we can't help but notice however that the underlying rationale is exactly the same (enabling list-initialization for a "value" parameter).

[P1997R1] proposes some changes to the core language in order to make array types copiable, assignable, etc. We are not proposing any changes to the language; however it's interesting to note that, if [P1997R1] is adopted, then using arrays as value types in containers and algorithms may become more widespread. (Right now, array types are usable in containers/algorithms in extremely limited ways.) The present proposal would then make it possible to use list-initialization in order to create arrays as the arguments of algorithmic-like functions, for instance like this:

```
// in principle, possible under P1997R1
std::vector<int[2]> v(42);
v.push_back({1, 2});

// in principle, possible under P1997R1 + this proposal
// (T defaults to int[2], second parameter is deduced as const int(&)[2],
// then aggregate initialization is selected)
std::ranges::fill(v, {123, 456});
```

[P2609R0] proposes an alias template (`indirect_value_t`) which is extremely similar to the `projected_value_t` alias proposed by this paper. `projected_value_t` could, in fact, be entirely reformulated in terms of `indirect_value_t` (cf. the discussion in § 5.9.1 Providing the projected value as an API).

§ 5. Design Decisions

§ 5.1. What is the rationale for the default chosen in each case?

We have tried to keep the choice of the default as much straightforward as possible, by following this set of rules/guidelines.

In the following, `T` is the template type parameter for the value-like parameter of each function.

§ 5.1.1. Consistent Container Erasure

These would simply default `T` to the "value type" of the container. We are proposing to change all these algorithms (for `basic_string`, `deque`, `forward_list`, `list`, and `vector`).

§ 5.1.2. Algorithms with an output iterator/range (where `T` belongs to that output)

For those we are proposing to use the `value_type` of the output iterators/range. (This may be `void`; see § 5.8 What about output iterators, whose value_type may be void?). The rationale is that the output iterator is going to give us the type which is going to be ultimately stored into it, and therefore that's the type to default to.

The algorithms in this category that we are proposing to change are:

- `fill_n`
- `replace_copy` (see § 5.1.4 The problem of establishing a default for replace_copy)
- `replace_copy_if`

§ 5.1.3. Algorithms with a (mutable) input iterator/range (where `T` belongs to the input)

Similarly, we're proposing to default the value-like type to the `value_type` of the input iterator/range, eventually after projection (in the case of range-based algorithms); see the discussion in § 5.9 The need of a custom projection type transformation.

The algorithms in this category that we are proposing to change are:

- `find`
- `count`
- `search_n`
- `replace`
- `replace_if`
- `replace_copy` (see § 5.1.4 The problem of establishing a default for replace_copy)
- `fill`
- `remove`
- `remove_copy`
- `lower_bound`
- `upper_bound`
- `equal_range`
- `binary_search`

§ 5.1.4. The problem of establishing a default for `replace_copy`

The non-range version of `replace_copy` has this signature in [algorithm.syn]:

```
template<class InputIterator, class OutputIterator, class T>
  constexpr OutputIterator replace_copy(InputIterator first, InputIterator last,
                                        OutputIterator result,
                                        const T& old_value, const T& new_value);
```

The issue with this signature is that there is only **one** template type parameter for both the old value and the new value. (This is probably an historical mistake; changing `replace_copy` to have two distinct template type parameters for the old value and the new value is out of scope for the present proposal.)

Now, if we were to pick a default for `T`, we could choose to default it to either the input iterator's value type, or to the output iterator's value type. Unfortunately both choices could lead to confusion and/or surprising behavior for end-users, who might expect the parameter type to be defaulted to the "other" value type, and thus accidentally introduce bugs.

Consider this example, from a discussion on the LEWG reflector:

```
std::vector<std::pair<int, int>> intpairs;
std::vector<std::pair<float, float>> floatpairs;

std::replace_copy(intpairs.begin(), intpairs.end(),
                  floatpairs.begin(),
                  { 1, 2 },
                  { 3.14, 3.14 }); // <-
```

If `T` is defaulted to the *input*'s value type, then the last parameter is actually a `std::pair<int, int>` equal to `{3, 3}`; the example has some (possibly unexpected) hidden data loss.

If `T` is instead defaulted to the *output*'s value type, one can build a very similar example:

```
std::replace_copy(floatpairs.begin(), floatpairs.end(),
                  intpairs.begin(),
                  { 3.14, 3.14 }, // <-
                  { 1, 2 });
```

Now the fourth parameter will be converted to `{3, 3}`, and again possibly change the program semantics in an unexpected way.

In conclusion: given no choice is *strongly* better than the other, and given that ultimately the problem is due to an API flaw of `replace_copy`, in R4 we are **not** proposing any change to the non-range version.

The range version of `replace_copy` does not suffer from this issue, because it features two different template type parameters, and therefore allowing for two different defaults (the old value belongs to the input range, the new value belongs to the output range). We are proposing to add the defaults to that version, chosen according to the above criteria.

§ 5.2. Numeric algorithms

Starting from the R6 revision of this paper, we are **not** proposing to change the numeric algorithms (in `<numeric>`).

Although such algorithms also have a value-like parameter (usually used a "starting point"), there does not seem to be consensus for this change: a LEWG telecon poll on 2022-08-02 had results 1/6/1/5/1.

Furthermore, additional discussion is needed in order to determine the correct default, which may not necessarily be the `value_type` of the iterators.

For these reasons, we are going to remove the numeric algorithms from the present proposal, and submit a separate one to address them specifically.

§ 5.3. Is this API compatible?

We are fairly certain that the proposed changes keep API compatibility.

Technically speaking, defaulting a template type parameter of a function template is a detectable change. Here's an example (many thanks to of Arthur O'Dwyer, who provided a much simpler one than ours):

```
1 template <typename A, typename B /* = A */> int f(A, B) { return 1; }
2
3 int f(int, int *) { return 2; }
4
5 int x = f(0, {0});
```

In this code `x` is equal to 2 (there is only one candidate). If we change line 1 so that `B` is defaulted to `A` (removing the comment marks), then that function template is able to provide a better candidate and `x` will be 1 instead.

We do not think this is actually a meaningful problem. For this to affect client code, such code must be performing a function call that somehow is including a Standard Library function during its overload resolution, and then selecting some other function as the best candidate. If that's the case, any change in the Standard Library (such as adding an overload) could affect the overload resolution and select a different candidate.

In a similar way, one can detect the presence of a defaulted template parameter via SFINAE. If we take a "prototype" version of an algorithmic function from the Standard Library that accepts two iterators and a "value":

```
template <typename I, typename T>
I algorithm(I, I, const T &);
```

then a call such as `algorithm(~~~, ~~~, {})` will fail to deduce `T`, as the corresponding function parameter constitutes a non-deduced context ([temp.deduct.type]/5.6). On the other hand, if we add a default to its value-like template type parameter (like this paper is proposing):

```
template <typename I, typename T = typename iterator_traits<I>::value_type>
I algorithm(I, I, const T &);
```

then we are going to *allow* deduction for the corresponding function parameter, and the call before may become well-formed.

In principle, this will mean that code using SFINAE or other similar techniques may change behavior. We think that this is extremely unlikely to be happening in practice; if such code exists, it's certainly a mistake, as it would always currently be failing deduction (given the current specification of the Standard Library algorithms).

In conclusion, although an API break is possible in principle, it's extremely unlikely to be happening in practice. What's more, the kind of proposed changes fall under what the Standard Library is allowed to do. [SD-8] in fact gives us the explicit permission to default template parameters for function templates in the Standard Library:

> Primarily, the standard reserves the right to:
>
> [...]
>
> - Add new default arguments to functions and templates

Based on this document, users are already not supposed to rely on the absence of such defaults.

§ 5.4. Is this ABI compatible?

This proposal deals with two kind of changes to function templates.

§ 5.4.1. Changes to consistent container erasure and non-range-based algorithms

Here we are simply proposing to default a template type parameter in the template parameter list of the related functions. This is entirely ABI compatibile (we're dealing with *function* templates, and not class templates).

§ 5.4.2. Changes to range-based algorithms

These changes require reordering the template argument list (see the discussion in § 5.5 Range-based algorithms). This is unfortunately not 100% ABI compatibile, as the instantiations of these functions would change their mangling.

With the reorderings proposed, we feel it's very unlikely that this incompatibility would cause a problem in practice. A user would have to call range-based algorithm using the same type for the template parameters that got reordered. For instance, for `std::ranges::remove`, one would need to use the same type as the projection type and for the value to remove (see § 7.4.6 [alg.remove]) in order to get a clash.

In the absence of such a type clash, then the problem would be averted: old code would keep using the old mangled name, and code compiled against the new version would use the new name. (The final binary may end up with multiple copies of

the instantiated function, under different symbols.)

There would still be the chance of an ABI break for user code using explicit template instantiations of one of these functions. The TU defining the instantiation would then be incompatible with any other TU that uses a different version of these functions (and merely declares the instantation).

We are not sure that user code is actually allowed to use explicit instantiations of Standard Library functions. Such a usage seems to violate the [SD-8] directives, specifically the right of the Standard Library to:

> - Make changes to existing interfaces in a fashion that will be backward compatible, if those interfaces are solely used to instantiate types and invoke functions. Implementation details (the primary name of a type, the implementation details for a function callable) may not be depended upon.
>
>   For example, we may change implementation details for standard function templates so that those become callable function objects. If user code only invokes that callable, the behavior is unchanged.

Cf. also the discussion in the § 5.6 What about user-specified template parameters? section.

§ 5.5. Range-based algorithms

The current specification in the Standard for range-based algorithms does not allow for a "straightforward" defaulting, like the non-range-based variants.

`ranges::find` has this synopsis in [algorithm.syn]:

```
template<input_iterator I, sentinel_for<I> S, class T, class Proj = identity>
  requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
  constexpr I find(I first, S last, const T& value, Proj proj = {});

template<input_range R, class T, class Proj = identity>
  requires indirect_binary_predicate<ranges::equal_to,
                                     projected<iterator_t<R>, Proj>, const T*>
  constexpr borrowed_iterator_t<R>
    find(R&& r, const T& value, Proj proj = {});
```

Both declarations do not allow for `T` to be defaulted given the ordering of template type parameters. `T` should be defaulted to something like `typename projected<I, Proj>::value_type` in the first version, and to `typename projected<iterator_t<R>, Proj>::value_type` in the second version. (Technically, neither; see the discussion in § 5.9 The need of a custom projection type transformation). This is not possible: the template parameter `Proj` appears *after* `T` in the template parameter list.

In order to fix this, we need to reorder the template parameters.

Some other algorithms need a slightly more elaborate change. Let's consider `ranges::fill`:

```
template<class T, output_iterator<const T&> O, sentinel_for<O> S>
  constexpr O fill(O first, S last, const T& value);

template<class T, output_range<const T&> R>
  constexpr borrowed_iterator_t<R> fill(R&& r, const T& value);
```

Here the output iterator `O` is defined in terms of `T`, making it impossible to default `T` in terms of `O` instead. The proposed solution in this case is reorder the template parameter list and move the constraint out of it, in a `requires` clause. This will keep the pre-existing semantics while allowing `T` to be defaulted.

To summarize, the changes that we are proposing to the range-based algorithms go in this direction: they move template parameters and/or constraints in order to be able to give a meaningful default type to the "value"-like function parameters.

We believe that [SD-8] allows us to do so under the interpretation that the exact spelling or ordering of template type arguments and constraints is part of the "implementation details" that users cannot depend upon. There will be no changes for

users who simply *call* these functions, letting the template arguments to be deduced.

§ 5.6. What about user-specified template parameters?

For the non-algorithmic changes (that is, the ones to consistent container erasure), we are not changing the order nor the meaning of the template parameters. User code that is calling such functions specifying the template type parameters explicitly is therefore not going to be affected.

For the algorithms, users are already *not allowed* to rely on the number and the order of deducible template parameters. [algorithms.requirements] / 15 says:

> The number and order of deducible template parameters for algorithm declarations are unspecified, except where explicitly stated otherwise.
> [Note 3: Consequently, an implementation can reject calls that specify an explicit template argument list. — end note]

Therefore, we do have the freedom not only of defaulting them, but also of reordering them when necessary.

Please note, however, that in spite of this paragraph in the Standard and of the SD-8 rules, the kind of changes we are proposing may be ABI incompatible. See § 5.4 Is this ABI compatible? for a discussion.

§ 5.7. Is the Standard Library allowed to reorder deducible template parameters of algorithms?

During the LEWG review of R6 of this paper in Kona a doubt has been raised regarding whether or not the Standard Library is allowed to reorder deducible template parameters. This reordering is necessary for range-based algorithms, as explained in § 5.5 Range-based algorithms.

[SD-8] does not explicitly state whether such a reordering is admissibile or not. Nonetheless, we argue that we have the freedom of doing so, for at least two reasons.

The first reason (discussed in § 5.6 What about user-specified template parameters?) is that users are already not allowed to rely on the order or the number of deducible template parameters for algorithms. This is stated in [algorithms.requirements] / 15:

> The number and order of deducible template parameters for algorithm declarations are unspecified, except where explicitly stated otherwise.
> [Note 3: Consequently, an implementation can reject calls that specify an explicit template argument list. — end note]

None of the algorithms that we propose to modify offers guarantees regarding the number and the order of their parameters; portable code can therefore make no assumptions in this regard.

The second reason (discussed in § 5.4 Is this ABI compatible? and § 5.5 Range-based algorithms) is that [SD-8] states:

> - Make changes to existing interfaces in a fashion that will be backward compatible, if those interfaces are solely used to instantiate types and invoke functions. Implementation details (the primary name of a type, the implementation details for a function callable) may not be depended upon.
>
>   For example, we may change implementation details for standard function templates so that those become callable function objects. If user code only invokes that callable, the behavior is unchanged.

We believe that the precise order and number of deducible template parameters for algorithms belong to the "implementation details". We are not proposing any behavioral change for code that simply calls an algorithmic function.

§ 5.8. What about output iterators, whose `value_type` may be `void`?

In this case, the changes proposed by this paper would not bring any effect; an user would still not be able to call an algorithm using an initializer list without also specifying the type to deduce.

§ 5.9. The need of a custom projection type transformation

Many range-based algorithms feature a projection function. Let's use `ranges::find` again as an example:

```
template<input_range R, class T, class Proj = identity>
  requires indirect_binary_predicate<ranges::equal_to,
                                     projected<iterator_t<R>, Proj>, const T*>
  constexpr borrowed_iterator_t<R>
    find(R&& r, const T& value, Proj proj = {});
```

For these algorithms `T` should not be defaulted to the `range_value_t` of the input range, but rather to the type obtained after applying the projection:

```
struct point { int x; int y; };

struct tagged_point
{ std::string tag; point p; };

std::vector<tagged_point> v;

auto i = std::ranges::find(v,
                           {1, 2}, // should be a point because of the projection
                           &tagged_point::p);
```

One could therefore conclude that the correct default for `find`'s `T` template parameter should be `typename std::projected<std::iterator_t<R>, Proj>::value_type`.

Unfortunately, as discussed by [P2322R6], this does not work in all cases; ranges yielding proxy iterators would end up using the wrong type as default.

If `Proj` is `std::identity` (the common case), and the range is e.g. `std::vector<bool>`, then `T` would *not* be defaulted to `bool` as one would expect, rather to `std::vector<bool>::reference`. This type is not copy-list constructible from `bool`, and therefore would make this code ill-formed:

```
std::vector<bool> v;
auto i = std::ranges::find(v, {true}); // ERROR
```

Another example (again brought forward by [P2322R6]) is the case of of `views::zip`. A `zip` over a range of `X` uses a `std::tuple<X&>` as its `range_reference_t` type, and therefore `std::tuple<X&>` would be the type obtained by the projection (again when using `std::identity`). This type is not useful at all as a default for `T`, as one cannot initialize such a tuple with, say, a prvalue `X` in a list initialization:

```
std::vector<int> v1, v2;
auto i = std::ranges::find(std::views::zip(v1, v2), {3, 4}); // ERROR
```

The problem here is that `std::projected` is meant to be used to constrain an algorithm based on the semantic operations that an algorithm needs to perform in its implementation. These operations are (normally) performed using projected references -- that is, invoking the projection on the range's `range_reference_t`.

The default for a value-like parameter does not however need reference semantics -- it needs value semantics, projecting a range's *value type*. Specifically, projecting `range_value_t<R> &`, as that's what's required from projections (and enforced by the `indirectly_regular_unary_invocable` concept). While for most "ordinary" ranges this would be the same as projecting the range's reference type, this would be different from ranges yielding proxy iterators.

In short, the correct default for `T` given a range `R` and a projection `Proj` should be:

`remove_cvref_t<invoke_result_t<Proj&, range_value_t<R>&>>`

With this, the default for a `std::vector<bool>` range would indeed be `bool`; and the default for a `zip` view over a range of `X` would be `std::tuple<X>` -- again assuming using `identity` as projection, i.e. the common case.

Technically speaking, in the general case, an projection may return different types when invoked with a `range_value_t<R>&` instead of a `range_reference_t`. This projection:

```
struct evil_projection {
    int operator()(const bool &) const;
    double operator()(bool &&) const;
};
```

used with a `std::vector<bool>` range yields `int` when projecting lvalue references to range values (i.e. `bool &`), and `double` when projecting range references (i.e. `std::vector<bool>::reference`). While legal, such kind of projections has very dubious value, and we do not believe they should constitute a problem in practice. (They would just be used to determine the default type for `T`; and such default type would still be subjected to the other constraints of the algorithms.)

§ 5.9.1. Providing the projected value as an API

While we could put the above solution to the projected value as an exposition-only type transformation, we are actually proposing to modify `<iterator>` in order to provide this facility to end-users, so that they can use it as well -- for instance, if they also want to default the value-like parameters of their own algorithms following the "lead" of the Standard Library itself.

To this end, we are proposing to add a `projected_value_t` alias template. This alias exposes the type transformation described above; it's the logical counterpart of `typename projected<I, Proj>::value_type`, obtained using `iter_value_t<I>&` instead of `iter_reference_t<I>`:

```
// Same as above, just using an indirectly_readable I rather than a range R
template<indirectly_readable I, indirectly_regular_unary_invocable<I> Proj>
  using projected_value_t = remove_cvref_t<invoke_result_t<Proj&, iter_value_t<I>&>>;
```

This allows us to reformulate the default for `T` as:

`projected_value_t<iterator_t<R>, Proj>`

which is the form found in the proposed wording.

[P2609R0] proposes an alias template extremely similar to `projected_value_t`, called `indirect_value_t`. The latter is a generalization of the former. In principle, following [P2609R0]'s adoption, `projected_value_t<I, Proj>` could be reformulated as `remove_cvref_t<indirect_value_t<projected<I, Proj>>>`, with absolutely no change in semantics.

§ 6. Implementation experience

A working prototype of the changes proposed by this paper, done on top of GCC 11, is available in this GCC branch on GitHub.

The HPX library has also implemented changes along the lines of the ones proposed by this paper.

§ 7. Technical Specifications

All the proposed changes are relative to [N4971].

§ 7.1. Feature testing macro

Add to the list in [version.syn]:

```
#define __cpp_lib_default_template_type_for_algorithm_values YYYYMML  // also in
    // <algorithm>, <ranges>,
```

```
    // <string>, <deque>, <list>, <forward_list>, <vector>
```

with the value specified as usual (year and month of adoption).

§ 7.2. Consistent Container Erasure

Modify [string.syn] and [string.erasure]:

```
template<class charT, class traits, class Allocator, class U = charT>
  constexpr typename basic_string<charT, traits, Allocator>::size_type
    erase(basic_string<charT, traits, Allocator>& c, const U& value);
```

Modify [deque.syn] and [deque.erasure]:

```
template<class T, class Allocator, class U = T>
  typename deque<T, Allocator>::size_type
    erase(deque<T, Allocator>& c, const U& value);
```

Modify [forwardlist.syn] and [forward.list.erasure]:

```
template<class T, class Allocator, class U = T>
  typename forward_list<T, Allocator>::size_type
    erase(forward_list<T, Allocator>& c, const U& value);
```

Modify [list.syn] and [list.erasure]:

```
template<class T, class Allocator, class U = T>
  typename list<T, Allocator>::size_type
    erase(list<T, Allocator>& c, const U& value);
```

Modify [vector.syn] and [vector.erasure]:

```
template<class T, class Allocator, class U = T>
  constexpr typename vector<T, Allocator>::size_type
    erase(vector<T, Allocator>& c, const U& value);
```

§ 7.3. Iterators

Modify the `<iterator>` header synopsis ([iterator.synopsis]):

```
// [projected], projected
template<indirectly_readable I, indirectly_regular_unary_invocable<I> Proj>
  struct projected; // freestanding

template<weakly_incrementable I, class Proj>
  struct incrementable_traits<projected<I, Proj>>; // freestanding


template<indirectly_readable I, indirectly_regular_unary_invocable<I> Proj>
  using projected_value_t = remove_cvref_t<invoke_result_t<Proj&, iter_value_t<I>&>>; // fr
```

Note to LWG / to the editor: this specification of `projected_value_t` could be changed to a completely equivalent one using the exposition-only alias template `indirect-value-t`. Please see the discussion in § 5.9.1 Providing the projected value as an API.

§ 7.4. Algorithms

Modify both the `<algorithm>` synopsis ([algorithm.syn]) as well as the algorithm signatures in each and every referenced section:

§ 7.4.1. [alg.find]

```
template<class InputIterator, class T = typename iterator_traits<InputIterator>::value_type
  constexpr InputIterator find(InputIterator first, InputIterator last,
                               const T& value);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<F
  ForwardIterator find(ExecutionPolicy&& exec,
                       ForwardIterator first, ForwardIterator last,
                       const T& value);
```

```
namespace ranges {
  template<input_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T = p
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr I find(I first, S last, const T& value, Proj proj = {});
  template<input_range R, class T, class Proj = identity, class T = projected_value_t<itera
    requires indirect_binary_predicate<ranges::equal_to,
                                       projected<iterator_t<R>, Proj>, const T*>
    constexpr borrowed_iterator_t<R>
      find(R&& r, const T& value, Proj proj = {});
}
```

§ 7.4.2. [alg.count]

```
template<class InputIterator, class T = typename iterator_traits<InputIterator>::value_type
  constexpr typename iterator_traits<InputIterator>::difference_type
    count(InputIterator first, InputIterator last, const T& value);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<F
  typename iterator_traits<ForwardIterator>::difference_type
    count(ExecutionPolicy&& exec,
          ForwardIterator first, ForwardIterator last, const T& value);
```

```
namespace ranges {
  template<input_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T = p
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr iter_difference_t<I>
      count(I first, S last, const T& value, Proj proj = {});
  template<input_range R, class T, class Proj = identity, class T = projected_value_t<itera
    requires indirect_binary_predicate<ranges::equal_to,
                                       projected<iterator_t<R>, Proj>, const T*>
    constexpr range_difference_t<R>
      count(R&& r, const T& value, Proj proj = {});
}
```

§ 7.4.3. [alg.search]

```
template<class ForwardIterator, class Size, class T = typename iterator_traits<ForwardItera
  constexpr ForwardIterator
    search_n(ForwardIterator first, ForwardIterator last,
             Size count, const T& value);
template<class ForwardIterator, class Size, class T = typename iterator_traits<ForwardItera
  constexpr ForwardIterator
    search_n(ForwardIterator first, ForwardIterator last,
             Size count, const T& value, BinaryPredicate pred);
template<class ExecutionPolicy, class ForwardIterator, class Size, class T = typename itera
  ForwardIterator
    search_n(ExecutionPolicy&& exec,
             ForwardIterator first, ForwardIterator last,
             Size count, const T& value);
template<class ExecutionPolicy, class ForwardIterator, class Size, class T = typename itera
          class BinaryPredicate>
  ForwardIterator
    search_n(ExecutionPolicy&& exec,
             ForwardIterator first, ForwardIterator last,
             Size count, const T& value,
             BinaryPredicate pred);
```

```
namespace ranges {
  template<forward_iterator I, sentinel_for<I> S, class T,
           class Pred = ranges::equal_to, class Proj = identity, class T = projected_value_
    requires indirectly_comparable<I, const T*, Pred, Proj>
    constexpr subrange<I>
      search_n(I first, S last, iter_difference_t<I> count,
               const T& value, Pred pred = {}, Proj proj = {});
  template<forward_range R, class T, class Pred = ranges::equal_to,
           class Proj = identity, class T = projected_value_t<iterator_t<R>, Proj>>
    requires indirectly_comparable<iterator_t<R>, const T*, Pred, Proj>
    constexpr borrowed_subrange_t<R>
      search_n(R&& r, range_difference_t<R> count,
               const T& value, Pred pred = {}, Proj proj = {});
}
```

§ 7.4.4. [alg.replace]

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr void replace(ForwardIterator first, ForwardIterator last,
                         const T& old_value, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<F
  void replace(ExecutionPolicy&& exec,
               ForwardIterator first, ForwardIterator last,
               const T& old_value, const T& new_value);
template<class ForwardIterator, class Predicate, class T = typename iterator_traits<Forward
  constexpr void replace_if(ForwardIterator first, ForwardIterator last,
                            Predicate pred, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator, class Predicate, class T = typename
  void replace_if(ExecutionPolicy&& exec,
```

```
                  ForwardIterator first, ForwardIterator last,
                  Predicate pred, const T& new_value);
```

```
namespace ranges {
  template<input_iterator I, sentinel_for<I> S, class T1, class T2, class Proj = identity,
    requires indirectly_writable<I, const T2&> &&
             indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*>
    constexpr I
      replace(I first, S last, const T1& old_value, const T2& new_value, Proj proj = {});
  template<input_range R, class T1, class T2, class Proj = identity, class T1 = projected_v
    requires indirectly_writable<iterator_t<R>, const T2&> &&
             indirect_binary_predicate<ranges::equal_to,
                                       projected<iterator_t<R>, Proj>, const T1*>
    constexpr borrowed_iterator_t<R>
      replace(R&& r, const T1& old_value, const T2& new_value, Proj proj = {});
  template<input_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T = p
           indirect_unary_predicate<projected<I, Proj>> Pred>
    requires indirectly_writable<I, const T&>
    constexpr I replace_if(I first, S last, Pred pred, const T& new_value, Proj proj = {});
  template<input_range R, class T, class Proj = identity, class T = projected_value_t<itera
           indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
    requires indirectly_writable<iterator_t<R>, const T&>
    constexpr borrowed_iterator_t<R>
      replace_if(R&& r, Pred pred, const T& new_value, Proj proj = {});
}
```

```
template<class InputIterator, class OutputIterator, class Predicate, class T = typename ite
  constexpr OutputIterator replace_copy_if(InputIterator first, InputIterator last,
                                           OutputIterator result,
                                           Predicate pred, const T& new_value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class Predicate, class T = typename iterator_traits<ForwardIterator2>::value_type>
  ForwardIterator2 replace_copy_if(ExecutionPolicy&& exec,
                                   ForwardIterator1 first, ForwardIterator1 last,
                                   ForwardIterator2 result,
                                   Predicate pred, const T& new_value);
```

```
namespace ranges {
template<class I, class O>
  using replace_copy_result = in_out_result<I, O>;

template<input_iterator I, sentinel_for<I> S, class T1, class T2,
         output_iterator<const T2&> O, class Proj = identity>
template<input_iterator I, sentinel_for<I> S, class O,
         class Proj = identity, class T1 = projected_value_t<I, Proj>, class T2 = iter_valu
  requires indirectly_copyable<I, O> &&
           indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T1*> &&
           output_iterator<O, const T2&>
  constexpr replace_copy_result<I, O>
    replace_copy(I first, S last, O result, const T1& old_value, const T2& new_value,
                 Proj proj = {});
template<input_range R, class T1, class T2, output_iterator<const T2&> O,
         class Proj = identity>
template<input_range R, class O, class Proj = identity,
         class T1 = projected_value_t<iterator_t<R>, Proj>, class T2 = iter_value_t<O>>
  requires indirectly_copyable<iterator_t<R>, O> &&
           indirect_binary_predicate<ranges::equal_to,
                                     projected<iterator_t<R>, Proj>, const T1*> &&
```

```
          output_iterator<O, const T2&>
  constexpr replace_copy_result<borrowed_iterator_t<R>, O>
    replace_copy(R&& r, O result, const T1& old_value, const T2& new_value,
                 Proj proj = {});

template<class I, class O>
  using replace_copy_if_result = in_out_result<I, O>;

template<input_iterator I, sentinel_for<I> S, class T, output_iterator<const T&> O, class O,
         class Proj = identity, indirect_unary_predicate<projected<I, Proj>> Pred>
  requires indirectly_copyable<I, O> && output_iterator<O, const T&>
  constexpr replace_copy_if_result<I, O>
    replace_copy_if(I first, S last, O result, Pred pred, const T& new_value,
                    Proj proj = {});
template<input_range R, class T, output_iterator<const T&> O, class O, class T = iter_value
         indirect_unary_predicate<projected<iterator_t<R>, Proj>> Pred>
  requires indirectly_copyable<iterator_t<R>, O> && output_iterator<O, const T&>
  constexpr replace_copy_if_result<borrowed_iterator_t<R>, O>
    replace_copy_if(R&& r, O result, Pred pred, const T& new_value,
                    Proj proj = {});
}
```

§ **7.4.5. [alg.fill]**

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value
  constexpr void fill(ForwardIterator first, ForwardIterator last, const T& value);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<F
  void fill(ExecutionPolicy&& exec,
            ForwardIterator first, ForwardIterator last, const T& value);
template<class OutputIterator, class Size, class T = typename iterator_traits<OutputIterato
  constexpr OutputIterator fill_n(OutputIterator first, Size n, const T& value);
template<class ExecutionPolicy, class ForwardIterator,
         class Size, class T = typename iterator_traits<ForwardIterator>::value_type>
  ForwardIterator fill_n(ExecutionPolicy&& exec,
                         ForwardIterator first, Size n, const T& value);
```

```
namespace ranges {
  template<class T, output_iterator<const T&> O, sentinel_for<O> S>
  template<class O, sentinel_for<O> S, class T = iter_value_t<O>>
    requires output_iterator<O, const T&>
    constexpr O fill(O first, S last, const T& value);
  template<class T, output_range<const T&> R>
  template<class R, class T = range_value_t<R>>
    requires output_range<R, const T&>
    constexpr borrowed_iterator_t<R> fill(R&& r, const T& value);
  template<class T, output_iterator<const T&> O>
  template<class O, class T = iter_value_t<O>>
    requires output_iterator<O, const T&>
    constexpr O fill_n(O first, iter_difference_t<O> n, const T& value);
}
```

§ **7.4.6. [alg.remove]**

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value
  constexpr ForwardIterator remove(ForwardIterator first, ForwardIterator last,
                                   const T& value);
template<class ExecutionPolicy, class ForwardIterator, class T = typename iterator_traits<F
```

```
  ForwardIterator remove(ExecutionPolicy&& exec,
                         ForwardIterator first, ForwardIterator last,
                         const T& value);
```

```
namespace ranges {
  template<permutable I, sentinel_for<I> S, class T, class Proj = identity, class T = proje
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr subrange<I> remove(I first, S last, const T& value, Proj proj = {});
  template<forward_range R, class T, class Proj = identity, class T = projected_value_t<ite
    requires permutable<iterator_t<R>> &&
             indirect_binary_predicate<ranges::equal_to,
                                       projected<iterator_t<R>, Proj>, const T*>
    constexpr borrowed_subrange_t<R>
      remove(R&& r, const T& value, Proj proj = {});
}
```

```
template<class InputIterator, class OutputIterator, class T = typename iterator_traits<Inpu
  constexpr OutputIterator
    remove_copy(InputIterator first, InputIterator last,
                OutputIterator result, const T& value);
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
         class T = typename iterator_traits<ForwardIterator1>::value_type>
  ForwardIterator2
    remove_copy(ExecutionPolicy&& exec,
                ForwardIterator1 first, ForwardIterator1 last,
                ForwardIterator2 result, const T& value);
```

```
namespace ranges {
  template<input_iterator I, sentinel_for<I> S, weakly_incrementable O, class T,
           class Proj = identity, class T = projected_value_t<I, Proj>>
    requires indirectly_copyable<I, O> &&
             indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
    constexpr remove_copy_result<I, O>
      remove_copy(I first, S last, O result, const T& value, Proj proj = {});
  template<input_range R, weakly_incrementable O, class T, class Proj = identity, class T =
    requires indirectly_copyable<iterator_t<R>, O> &&
             indirect_binary_predicate<ranges::equal_to,
                                       projected<iterator_t<R>, Proj>, const T*>
    constexpr remove_copy_result<borrowed_iterator_t<R>, O>
      remove_copy(R&& r, O result, const T& value, Proj proj = {});
}
```

§ 7.4.7. [alg.binary.search]

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr ForwardIterator
    lower_bound(ForwardIterator first, ForwardIterator last,
                const T& value);
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
```

```
  constexpr ForwardIterator
    lower_bound(ForwardIterator first, ForwardIterator last,
                const T& value, Compare comp);
```

```
namespace ranges {
  template<forward_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T =
           indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
    constexpr I lower_bound(I first, S last, const T& value, Comp comp = {},
                            Proj proj = {});
  template<forward_range R, class T, class Proj = identity, class T = projected_value_t<ite
           indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
             ranges::less>
    constexpr borrowed_iterator_t<R>
      lower_bound(R&& r, const T& value, Comp comp = {}, Proj proj = {});
}
```

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr ForwardIterator
    upper_bound(ForwardIterator first, ForwardIterator last,
                const T& value);
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr ForwardIterator
    upper_bound(ForwardIterator first, ForwardIterator last,
                const T& value, Compare comp);
```

```
namespace ranges {
  template<forward_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T =
           indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
    constexpr I upper_bound(I first, S last, const T& value, Comp comp = {}, Proj proj = {}
  template<forward_range R, class T, class Proj = identity, class T = projected_value_t<ite
           indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
             ranges::less>
    constexpr borrowed_iterator_t<R>
      upper_bound(R&& r, const T& value, Comp comp = {}, Proj proj = {});
}
```

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last,
                const T& value);
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value_
  constexpr pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last,
                const T& value, Compare comp);
```

```
namespace ranges {
  template<forward_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T =
           indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
    constexpr subrange<I>
      equal_range(I first, S last, const T& value, Comp comp = {}, Proj proj = {});
  template<forward_range R, class T, class Proj = identity, class T = projected_value_t<ite
           indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
             ranges::less>
```

```
    constexpr borrowed_subrange_t<R>
      equal_range(R&& r, const T& value, Comp comp = {}, Proj proj = {});
}
```

```
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value
  constexpr bool
    binary_search(ForwardIterator first, ForwardIterator last,
                  const T& value);
template<class ForwardIterator, class T = typename iterator_traits<ForwardIterator>::value
  constexpr bool
    binary_search(ForwardIterator first, ForwardIterator last,
                  const T& value, Compare comp);
```

```
namespace ranges {
  template<forward_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T =
           indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ranges::less>
    constexpr bool binary_search(I first, S last, const T& value, Comp comp = {},
                                 Proj proj = {});
  template<forward_range R, class T, class Proj = identity, class T = projected_value_t<ite
           indirect_strict_weak_order<const T*, projected<iterator_t<R>, Proj>> Comp =
             ranges::less>
    constexpr bool binary_search(R&& r, const T& value, Comp comp = {},
                                 Proj proj = {});
}
```

§ 7.4.8. [alg.fold]

```
template<input_iterator I, sentinel_for<I> S, class T = iter_value_t<I>, indirectly-binary-
  constexpr auto ranges::fold_left(I first, S last, T init, F f);

template<input_range R, class T = range_value_t<R>, indirectly-binary-left-foldable<T, iter
  constexpr auto ranges::fold_left(R&& r, T init, F f);
```

```
template<bidirectional_iterator I, sentinel_for<I> S, class T = iter_value_t<I>,
         indirectly-binary-right-foldable<T, I> F>
  constexpr auto ranges::fold_right(I first, S last, T init, F f);
template<bidirectional_range R, class T = range_value_t<R>,
         indirectly-binary-right-foldable<T, iterator_t<R>> F>
  constexpr auto ranges::fold_right(R&& r, T init, F f);
```

```
template<input_iterator I, sentinel_for<I> S, class T = iter_value_t<I>,
         indirectly-binary-left-foldable<T, I> F>
  constexpr see below ranges::fold_left_with_iter(I first, S last, T init, F f);
template<input_range R, class T = range_value_t<R>, indirectly-binary-left-foldable<T, iter
  constexpr see below ranges::fold_left_with_iter(R&& r, T init, F f);
```

§ 7.4.9. [alg.contains]

```
template<input_iterator I, sentinel_for<I> S, class T, class Proj = identity, class T = pro
  requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>, const T*>
  constexpr bool contains(I first, S last, const T& value, Proj proj = {});

template<input_range R, class T, class Proj = identity, class T = projected_value_t<iterato
  requires indirect_binary_predicate<ranges::equal_to, projected<iterator_t<R>, Proj>, cons
  constexpr bool contains(R&& r, const T& value, Proj proj = {});
```

§ 8. Acknowledgements

Thanks to KDAB for supporting this work.

Special thanks to Stephan T. Lavavej for confirming that the omission of a default argument from the `erase()` functions was not a deliberate design choice of the consistent container erasure proposal. Thanks to Arthur O'Dwyer for the insightful discussions. Thanks to Will Wray for the discussions regarding supporting array types and the mentions of related paper work. Thanks to Michał Dominiak for pointing out the possible ABI issues but still encouraging to bring the paper forward. Thanks to Tim Song for raising the problem of establishing the default type in the presence of projections, and to Barry Revzin for the excellent analysis in [P2322R6].

All remaining errors are ours and ours only.

§ References

§ Informative References

[N4971]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4971.pdf

[P1997R1]
Krystian Stasiowski; Theodoric Stier. *Relaxing Restrictions on Arrays*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1997r1.pdf

[P2218R0]
Marc Mutz. *More flexible optional::value_or()*. URL: http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2218r0.pdf

[P2248-GCC]
Giuseppe D'Angelo. *P2248 prototype implementation*. URL: https://github.com/dangelog/gcc/tree/std-proposals

[P2248-HPX]
Implementation of P2248 in HPX. URL: https://github.com/STEllAR-GROUP/hpx/pull/5675

[P2302R4]
Christopher Di Bella. *std::ranges::contains*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2302r4.html

[P2322R6]
Barry Revzin. *ranges::fold*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2322r6.html

[P2609R0]
John Eivind Helset. *Relaxing Ranges Just A Smidge*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2609r0.html

[SD-8]
Titus Winter. *SD-8: Standard Library Compatibility*. URL: https://isocpp.org/std/standing-documents/sd-8-standard-library-compatibility